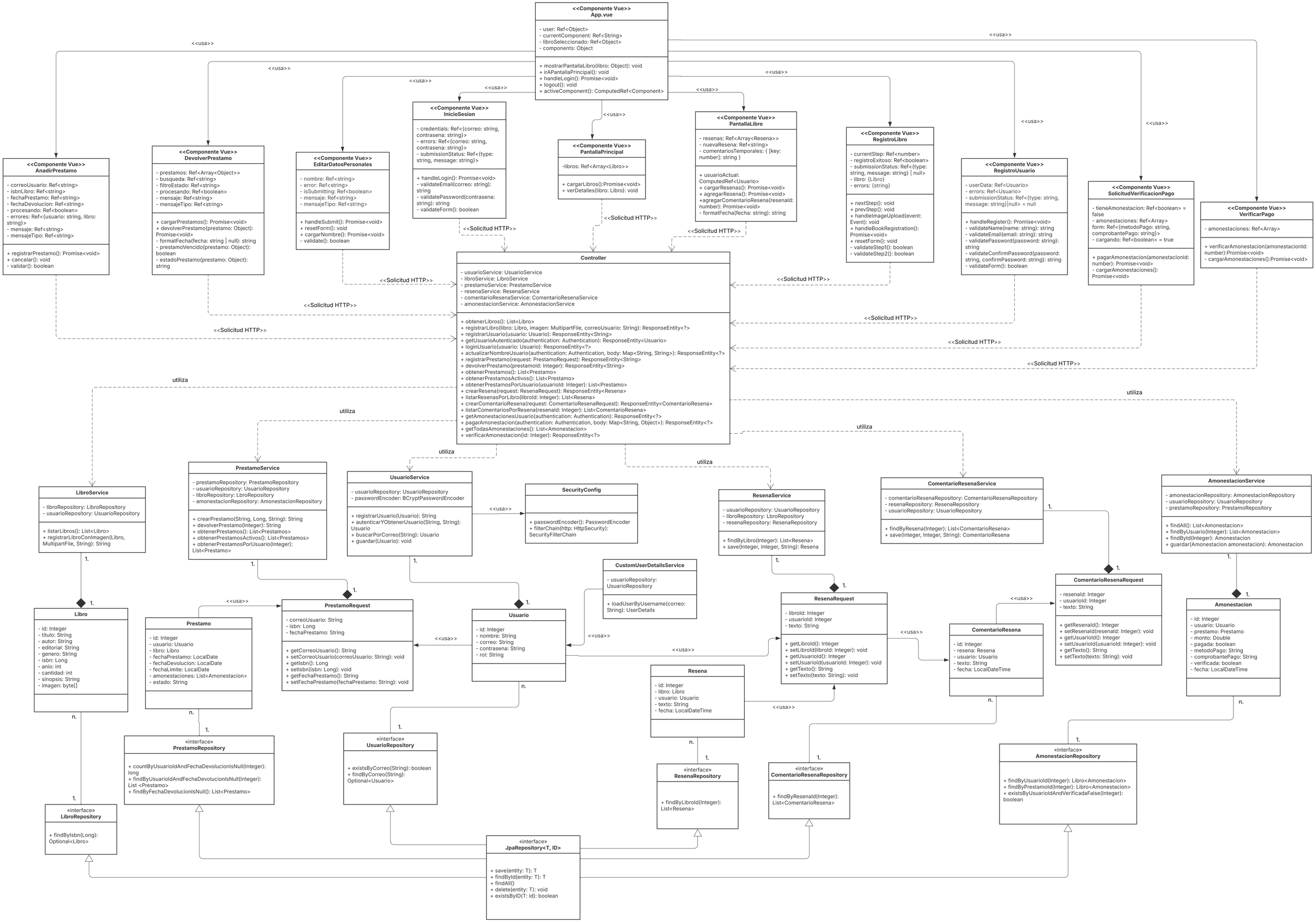
<<Componente Vue>>
App.vue
- user: Ref<Object>
- currentComponent: Ref<String>
- libroSeleccionado: Ref<Object>
- components: Object
+ mostrarPantallaLibro(libro: Object): void
+ irAPantallaPrincipal(): void
+ handleLogin(): Promise<void>
+ logout(): void
+ activeComponent(): ComputedRef<Component>
<<Componente Vue>>
AnadirPrestamo
- correoUsuario: Ref<string>
- isbnLibro: Ref<string>
- fechaPrestamo: Ref<string>
- fechaDevolucion: Ref<string>
- procesando: Ref<boolean>
- errores: Ref<{usuario: string, libro: string}>
- mensaje: Ref<string>
- mensajeTipo: Ref<string>
+ registrarPrestamo(): Promise<void>
+ cancelar(): void
- validar(): boolean
<<Componente Vue>>
DevolverPrestamo
- prestamos: Ref<Array<Object>>
- busqueda: Ref<string>
- filtroEstado: Ref<string>
- procesando: Ref<boolean>
- mensaje: Ref<string>
- mensajeTipo: Ref<string>
+ cargarPrestamos(): Promise<void>
+ devolverPrestamo(prestamo: Object): Promise<void>
- formatFecha(fecha: string | null): string
- prestamoVencido(prestamo: Object): boolean
- estadoPrestamo(prestamo: Object): string
<<Componente Vue>>
EditarDatosPersonales
- nombre: Ref<string>
- error: Ref<string>
- isSubmitting: Ref<boolean>
- mensaje: Ref<string>
- mensajeTipo: Ref<string>
+ handleSubmit(): Promise<void>
+ resetForm(): void
+ cargarNombre(): Promise<void>
- validate(): boolean
<<Componente Vue>>
InicioSesion
- credentials: Ref<{correo: string, contrasena: string}>
- errors: Ref<{correo: string, contrasena: string}>
- submissionStatus: Ref<{type: string, message: string}>
+ handleLogin(): Promise<void>
- validateEmail(correo: string): string
- validatePassword(contrasena: string): string
- validateForm(): boolean
<<Componente Vue>>
PantallaPrincipal
-libros: Ref<Array<Libro>>
+ cargarLibros():Promise<void>
+ verDetalles(libro: Libro): void
<<Componente Vue>>
PantallaLibro
- resenas: Ref<Array<Resena>>
- nuevaResena: Ref<string>
- comentariosTemporales: { [key: number]: string }
+ usuarioActual: ComputedRef<Usuario>
+ cargarResenas(): Promise<void>
+ agregarResena(): Promise<void>
+agregarComentarioResena(resenaId: number): Promise<void>
- formatFecha(fecha: string): string
<<Componente Vue>>
RegistroLibro
- currentStep: Ref<number>
- registroExitoso: Ref<boolean>
- submissionStatus: Ref<{type: string, message: string} | null>
- libro: {Libro}
- errors: {string}
+ nextStep(): void
+ prevStep(): void
+ handleImageUpload(event: Event): void
+ handleBookRegistration(): Promise<void>
+ resetForm(): void
- validateStep1(): boolean
- validateStep2(): boolean
<<Componente Vue>>
RegistroUsuario
- userData: Ref<Usuario>
- errors: Ref<Usuario>
- submissionStatus: Ref<{type: string, message: string}|null> = null
+ handleRegister(): Promise<void>
- validateName(name: string): string
- validateEmail(email: string): string
- validatePassword(password: string): string
- validateConfirmPassword(password: string, confirmPassword: string): string
- validateForm(): boolean
<<Componente Vue>>
SolicitudVerificacionPago
- tieneAmonestacion: Ref<boolean> = false
- amonestaciones: Ref<Array>
form: Ref<{metodoPago: string, comprobantePago: string}>
- cargando: Ref<boolean> = true
+ pagarAmonestacion(amonestacionId: number): Promise<void>
- cargarAmonestaciones(): Promise<void>
<<Componente Vue>>
VerificarPago
- amonestaciones: Ref<Array>
+ verificarAmonestacion(amonestacionId: number):Promise<void>
- cargarAmonestaciones():Promise<void>
<<usa>>
<<Solicitud HTTP>>
Controller
- usuarioService: UsuarioService
- libroService: LibroService
- prestamoService: PrestamoService
- resenaService: ResenaService
- comentarioResenaService: ComentarioResenaService
- amonestacionService: AmonestacionService
+ obtenerLibros(): List<Libro>
+ registrarLibro(libro: Libro, imagen: MultipartFile, correoUsuario: String): ResponseEntity<?>
+ registrarUsuario(usuario: Usuario): ResponseEntity<String>
+ getUsuarioAutenticado(authentication: Authentication): ResponseEntity<Usuario>
+ loginUsuario(usuario: Usuario): ResponseEntity<?>
+ actualizarNombreUsuario(authentication: Authentication, body: Map<String, String>): ResponseEntity<?>
+ registrarPrestamo(request: PrestamoRequest): ResponseEntity<String>
+ devolverPrestamo(prestamoId: Integer): ResponseEntity<String>
+ obtenerPrestamos(): List<Prestamo>
+ obtenerPrestamosActivos(): List<Prestamo>
+ obtenerPrestamosPorUsuario(usuarioId: Integer): List<Prestamo>
+ crearResena(request: ResenaRequest): ResponseEntity<Resena>
+ listarResenasPorLibro(libroId: Integer): List<Resena>
+ crearComentarioResena(request: ComentarioResenaRequest): ResponseEntity<ComentarioResena>
+ listarComentariosPorResena(resenaId: Integer): List<ComentarioResena>
+ getAmonestacionesUsuario(authentication: Authentication): ResponseEntity<?>
+ pagarAmonestacion(authentication: Authentication, body: Map<String, Object>): ResponseEntity<?>
+ getTodasAmonestaciones(): List<Amonestacion>
+ verificarAmonestacion(id: Integer): ResponseEntity<?>
utiliza
LibroService
- libroRepository: LibroRepository
- usuarioRepository: UsuarioRepository
+ listarLibros(): List<Libro>
+ registrarLibroConImagen(Libro, MultipartFile, String): String
PrestamoService
- prestamoRepository: PrestamoRepository
- usuarioRepository: UsuarioRepository
- libroRepository: LbroRepository
- amonestacionRepository: AmonestacionRepository
+ crearPrestamo(String, Long, String): String
+ devolverPrestamo(Integer): String
+ obtenerPrestamos(): List<Prestamos>
+ obtenerPrestamosActivos(): List<Prestamos>
+ obtenerPrestamosPorUsuario(Integer): List<Prestamo>
UsuarioService
- usuarioRepository: UsuarioRepository
- passwordEncoder: BCryptPasswordEncoder
+ registrarUsuario(Usuario): String
+ autenticarYObtenerUsuario(String, String): Usuario
+ buscarPorCorreo(String): Usuario
+ guardar(Usuario): void
SecurityConfig
+ passwordEncoder(): PasswordEncoder
+ filterChain(http: HttpSecurity): SecurityFilterChain
ResenaService
- usuarioRepository: UsuarioRepository
- libroRepository: LbroRepository
- resenaRepository: ResenaRepository
+ findByLibro(Integer): List<Resena>
+ save(Integer, Integer, String): Resena
ComentarioResenaService
- comentarioResenaRepository: ComentarioResenaRepository
- resenaRepository: ResenaRepository
- usuarioRepository: UsuarioRepository
+ findByResena(Integer): List<ComentarioResena>
+ save(Integer, Integer, String): ComentarioResena
AmonestacionService
- amonestacionRepository: AmonestacionRepository
- usuarioRepository: UsuarioRepository
- prestamoRepository: PrestamoRepository
+ findAll(): List<Amonestacion>
+ findByUsuario(Integer): List<Amonestacion>
+ findById(Integer): Amonestacion
+ guardar(Amonestacion amonestacion): Amonestacion
1.
Libro
- id: Integer
- título: String
- autor: String
- editorial: String
- genero: String
- isbn: Long
- anio: int
- cantidad: int
- sinopsis: String
- imagen: byte[]
Prestamo
- id: Integer
- usuario: Usuario
- libro: Libro
- fechaPrestamo: LocalDate
- fechaDevolucion: LocalDate
- fechaLimite: LocalDate
- amonestaciones: List<Amonestacion>
- estado: String
PrestamoRequest
- correoUsuario: String
- isbn: Long
- fechaPrestamo: String
+ getCorreoUsuario(): String
+ setCorreoUsuario(correoUsuario: String): void
+ getIsbn(): Long
+ setIsbn(Isbn: Long): void
+ getFechaPrestamo(): String
+ setFechaPrestamo(fechaPrestamo: String): void
Usuario
- id: Integer
- nombre: String
- correo: String
- contrasena: String
- rol: String
CustomUserDetailsService
- usuarioRepository: UsuarioRepository
+ loadUserByUsername(correo: String): UserDetails
ResenaRequest
- libroId: Integer
- usuarioId: Integer
- texto: String
+ getLibroId(): Integer
+ setLibroId(libroId: Integer): void
+ getUsuarioId(): Integer
+ setUsuarioId(usuarioId: Integer): void
+ getTexto(): String
+ setTexto(texto: String): void
Resena
- id: Integer
- libro: Libro
- usuario: Usuario
- texto: String
- fecha: LocalDateTime
ComentarioResena
- id: Integer
- resena: Resena
- usuario: Usuario
- texto: String
- fecha: LocalDateTime
ComentarioResenaRequest
- resenaId: Integer
- usuarioId: Integer
- texto: String
+ getResenaId(): Integer
+ setResenaId(resenaId: Integer): void
+ getUsuarioId(): Integer
+ setUsuarioId(usuarioId: Integer): void
+ getTexto(): String
+ setTexto(texto: String): void
Amonestacion
- id: Integer
- usuario: Usuario
- prestamo: Prestamo
- monto: Double
- pagada: boolean
- metodoPago: String
- comprobantePago: String
- verificada: boolean
- fecha: LocalDateTime
n.
«interface»
LibroRepository
+ findByIsbn(Long): Optional<Libro>
«interface»
PrestamoRepository
+ countByUsuarioIdAndFechaDevolucionIsNull(Integer): long
+ findByUsuarioIdAndFechaDevolucionIsNull(Integer): List <Prestamo>
+ findByFechaDevolucionIsNull(): List<Prestamo>
«interface»
UsuarioRepository
+ existsByCorreo(String): boolean
+ findByCorreo(String): Optional<Usuario>
«interface»
ResenaRepository
+ findByLibroId(Integer): List<Resena>
«interface»
ComentarioResenaRepository
+ findByResenaId(Integer): List<ComentarioResena>
«interface»
AmonestacionRepository
+ findByUsuarioId(Integer): Libro<Amonestacion>
+ findByPrestamoId(Integer): Libro<Amonestacion>
+ existsByUsuarioIdAndVerificadaFalse(Integer): boolean
«interface»
JpaRepository<T, ID>
+ save(entity: T): T
+ findById(entity: T): T
+ findAll()
+ delete(entity: T): void
+ existsByID(T: id): boolean